Ultimate AI Financial Advisor Agent Platform Blueprint

Complete Product, Architecture, AI, Data, and Engineering Specification
Updated May 2026 — reflects multi-repo implementation

This document specifies a world-class AI-powered personal financial advisor platform. The target outcome is an intelligent financial co-pilot for tracking investments, analysing wealth, monitoring debt, optimising taxes, projecting retirement scenarios, and generating personalised recommendations. The application supports South African financial structures while remaining extensible globally.

Implemented as four repositories: *financial-copilot-api* (system of record), *financial-copilot-ai-orchestrator* (LangGraph AI), *financial-copilot-web* (Next.js dashboard), and *financial-copilot-infra* (Azure IaC and local compose).

1. Product Vision

Build a premium-grade AI wealth management platform combining financial planning, investment tracking, AI reasoning, tax optimisation, debt optimisation, retirement modelling, and behavioural finance guidance. The experience should feel like a personal CFO, investment analyst, retirement planner, and AI co-pilot in one product.

2. Platform Architecture (Current)

Request flow (local and cloud):
Browser → *financial-copilot-web* (:3000) → *financial-copilot-ai-orchestrator* (:8000) → *financial-copilot-api* (:8080) → PostgreSQL (:5432)

Responsibility boundaries
• **financial-copilot-api** — Spring Boot portfolio service; owns PostgreSQL, Flyway migrations, financial calculations, persistence, auth (future), business rules, versioned REST at `/api/v1/*`.
• **financial-copilot-ai-orchestrator** — FastAPI + LangGraph; prompt management, multi-agent routing, tool calling, AI explanations. **Does not connect to PostgreSQL.** All figures are read from the portfolio API.
• **financial-copilot-web** — Next.js 14 dashboard; calls API and orchestrator via `NEXT_PUBLIC_API_URL` and `NEXT_PUBLIC_AI_URL`.
• **financial-copilot-infra** — Bicep modules (Container Apps, PostgreSQL, Redis, Key Vault, Blob, Azure OpenAI), environment parameters, and full-stack `docker-compose.yml` for local development.

```
Local ports
Web 3000 · AI Orchestrator 8000 · Portfolio API 8080 · PostgreSQL 5432
```

3. Core User Features

Portfolio tracking · Multi-currency (ZAR/USD) · Net worth and health score · Asset allocation and regional exposure · AI financial chat · CSV imports (EasyEquities, bank, bond, RA samples) · Demo data seeding · Dashboard analytics · Bond optimisation (planned) · Retirement/TFSA/RA logic (planned) · Scenario simulations (agent-routed) · Monthly reports (agent-routed) · Risk analysis (agent-routed)

4. Technology Stack

Frontend: Next.js 14, React, TypeScript, Tailwind CSS
Portfolio API: Java 21, Spring Boot 3.3, Maven, Flyway, PostgreSQL 16
AI orchestration: Python 3.11, FastAPI, LangGraph, LangChain
LLM providers: Gemini 2.5 Flash (local dev via Google AI Studio); Azure OpenAI (production on Azure Container Apps)
Database: PostgreSQL (owned exclusively by portfolio API)
Cache (infra): Redis (Azure, provisioned in Bicep)
Charts: Recharts (web)
Hosting: Azure Container Apps, Azure Container Registry, Static Web Apps (web)
Observability: Log Analytics, Spring Actuator (`/actuator/health`)
CI/CD: GitHub Actions per repository (build, push to ACR, deploy ACA)

5. AI Agent Architecture (Implemented)

The AI layer is agentic, not a simple chatbot. LangGraph orchestrates a stateful workflow with planner-based routing and optional portfolio context loaded from the portfolio API.

Specialist agents (routing keywords → agent):
• Planner — routes user intent
• Portfolio Analysis — holdings, net worth, allocation, ETFs
• Tax — CGT, RA deductions, SA tax topics
• Bond Optimisation — mortgage, home loan, payoff
• Risk Analysis — volatility, drawdown
• Recommendation — personalised suggestions
• Report Generation — summaries and documents
• Scenario — what-if and simulations
• General — fallback conversational agent

```
Orchestrator API (financial-copilot-ai-orchestrator):
GET /health · GET /health/dependencies · POST /api/v1/chat · GET /api/v1/agents
```

LLM configuration
• Local: `LLM_PROVIDER=gemini`, `GOOGLE_API_KEY`, `GEMINI_MODEL=gemini-2.5-flash`
• Production: `LLM_PROVIDER=azure_openai` with `AZURE_OPENAI_*` secrets in GitHub Actions / Container Apps

6. Portfolio API Surface (v1)

```
GET /api/v1/dashboard — net worth, health score, allocation, top holdings
GET|POST /api/v1/accounts · GET|POST /api/v1/accounts/{id}/holdings
POST /api/v1/demo/seed?reset=true — demo portfolio data (reseed)
/api/v1/imports/* — CSV upload, preview, commit, mapping profiles
Swagger UI: http://localhost:8080/swagger-ui.html
```

7. Financial Calculation Engine

Calculations run in the portfolio API (not in the AI service). Targets include: compound growth, CAGR, ETF overlap, allocation analysis, risk scoring, drawdown, retirement modelling, bond amortisation, tax estimation, inflation-adjusted projections, and currency exposure. AI agents explain and narrate results; deterministic rules precede LLM reasoning.

8. South African Finance Logic

TFSA annual and lifetime limits · RA deduction calculations · SA tax brackets · Capital gains tax estimation · USD dividend withholding · Estate tax awareness · ZAR/USD allocation · SA bond and interest-rate sensitivity. Account types in schema: TFSA, RA, ZAR_BROKER, USD_BROKER, BANK, CASH.

9. Database Design

PostgreSQL schema (Flyway): users, risk_profiles, accounts, holdings, transactions, bond_accounts, retirement_accounts, asset_prices, exchange_rates, goals, recommendations, cash_flows, monthly_snapshots, tax_estimates, import jobs. Migrations: `backend/portfolio-service/src/main/resources/db/migration/`

10. File Import System

CSV import pipeline with upload, preview, validation, commit, and mapping profiles. Sample files for EasyEquities holdings/transactions, bank statements, bond statements, and RA statements. PDF parsing and broker API integrations remain on the roadmap.

11. Dashboard & Web Client

Next.js dashboard (*financial-copilot-web*) displays portfolio overview via the API. Targets: net worth graph, asset allocation, growth charts, retirement projection, debt payoff, currency exposure, contribution trends, financial health score, and cash flow analysis.

12. AI Prompt System & Guardrails

System prompt: educational analysis, simulations, calculations, trade-offs, and risk assessment — not licensed financial advice. Always explain reasoning, present options, consider taxes/risk/liquidity/diversification, use plain language, and reference user data from the portfolio API when available. Never guarantee returns or pose as a licensed advisor.

13. Security Requirements

Encrypted data at rest and in transit · Role-based access (planned) · Audit logging · JWT/OAuth2 (planned) · 2FA (planned) · Azure Key Vault for secrets · Secure API gateway · Rate limiting · Anomaly detection (roadmap). AI orchestrator has no direct database access, reducing blast radius.

14. Deployment Architecture

Azure (Bicep — financial-copilot-infra):
Log Analytics · Key Vault · Blob Storage · PostgreSQL Flexible Server · Redis · Azure OpenAI · Container Apps (portfolio API + AI orchestrator) · ACR

Per-repo CI/CD:
• *financial-copilot-api* — deploy portfolio-service container
• *financial-copilot-ai-orchestrator* — build image, push ACR, deploy ACA (sets `LLM_PROVIDER=azure_openai`)
• *financial-copilot-web* — Static Web Apps or container
• *financial-copilot-infra* — infrastructure deployment workflows

```
Local development:
API: scripts/run-backend.ps1 (Docker Postgres + mvn spring-boot:run)
AI: scripts/run.ps1 or docker compose up in orchestrator repo
Web: npm run dev
Full stack: financial-copilot-infra/scripts/local-up.ps1
```

15. Implementation Status & Roadmap

Delivered (MVP foundation):
• Multi-repo platform with clear boundaries
• Portfolio API with dashboard, accounts, holdings, demo seed
• PostgreSQL + Flyway schema
• CSV import API and samples
• LangGraph orchestrator with multi-agent routing
• Dual LLM providers (Gemini dev / Azure OpenAI prod)
• Next.js dashboard shell
• Docker Compose for Postgres and orchestrator
• GitHub Actions deploy to Azure Container Apps (orchestrator)

Next phases:
• Auth (OAuth2/JWT) and multi-user isolation
• Bond optimiser and retirement planner modules in API
• Full dashboard charts and chat UI in web
• AI memory and personalisation
• Monte Carlo and advanced analytics
• Broker API integrations and mobile app

16. Future Advanced Capabilities

AI-generated monthly reports · Voice assistant · Autonomous rebalancing suggestions · Behavioural coaching · Macro-economic awareness · Automated document ingestion · Broker API integrations · Mobile app · AI tax assistant · Financial goal gamification

17. Engineering Standards

Clean architecture · Microservice-friendly repos · Extensible financial models · Strong observability · Premium UI/UX · Institution-grade analytics · Scalable AI orchestration · Secure financial data handling · Cloud-native Azure deployment. The system must feel premium, intelligent, trustworthy, and highly analytical.

Sample user financial context (reference)
Net salary R60,000 · Emergency fund R100,000 · Bond R1,737,341.55 @ 9.4% · TFSA R3,000/mo · ZAR ETF R7,000/mo · USD ETF R10,000/mo · RA R6,750/mo · Bond overpayment R7,000/mo

End of Specification — Ready for AI Agent Consumption · May 2026